

A Mathematical Model of Contextual Capabilities

Formally Verified Static Capability Tracking in λ^{CC}

Fengyun Liu
TypeScope

Abstract

Capability-based security offers a principled way to enforce least privilege, but three gaps remain. Traditional capability systems such as the object-capability model typically do not support static checking of capability usage. Meanwhile, manual propagation of capabilities through every interface incurs substantial boilerplate when multiple fine-grained capabilities are involved. Finally, existing capability systems do not always treat capabilities as abstract interfaces to enable simulation and testing.

We present λ^{CC} , a minimal calculus of *contextual capabilities*: capabilities that are both statically tracked and selectively supplied by the surrounding calling context. The development is mechanized in Coq. Although λ^{CC} is intentionally minimal, it provides a mathematical core for designing secure programming languages with statically checked capability propagation, authority attenuation, and safe execution of untrusted code.

Keywords: capability-based security, formal verification, capability systems, lambda calculus

Contents

1	Introduction	3
1.1	Research Gap	3
1.2	Contextual Capabilities	3
1.3	Contribution	3
2	Contextual Capabilities	4
3	The λ^{CC} Calculus	5
3.1	Semantics	5
3.2	Type System and Capability Tracking	5
3.3	Security Properties	7
4	Soundness Theorem	7
4.1	Main Theorem	7
4.2	Why This Is Enough	7
4.3	Proof Strategy	7
4.4	The Fundamental Lemma	7
4.5	Mechanization	8
5	Case Study: Confining LLM-Generated Code	8
5.1	The Problem	8
5.2	The Jo Design	8
5.3	Why This Confines Authority	9

6	Related Work	10
6.1	Capability Disciplines Without Static Tracking	10
6.2	Static Requirement Tracking	10
6.3	Verified Capability Reasoning	10
7	Conclusion	11

1 Introduction

Software systems increasingly execute code that is neither fully trusted nor fully reviewed before deployment. Some of this code comes from plugins or user-defined scripts. Increasingly, it is generated on demand by large language models. In all of these settings, the security problem is the same: how do we ensure that code receives exactly the authority it needs, and no more?

This is a *least-privilege* problem: authority should be explicit, bounded, and reviewable rather than ambient [7]. Capability-based security gives the right general answer to least privilege and confused-deputy problems [8], but past capability models and capability-oriented languages such as E and Joe-E leave several important problems unresolved [4, 5].

This paper presents a mechanized mathematical model of *contextual capabilities*. The main purpose of the model is to provide a foundation for designing new secure programming languages in which authority is explicit, statically tracked, and controlled by program context.

1.1 Research Gap

There are three major limitations in past capability models that affect the usability of secure programming languages.

First, a capability system should support *static checking of capability usage*. It should be possible to read a function’s type signature as a trustworthy upper bound on the capabilities it may use. The object-capability model [2, 3] gives a strong design discipline, but it does not by itself provide a compact type-level account of capability use that can be checked compositionally.

Second, a capability system should facilitate the *propagation of fine-grained capabilities*. Capability-based programs do not carry a single coarse permission; they pass user-scoped services, output channels, loggers, tool bindings, and other narrow capabilities through higher-order code. Without language support, manual propagation through every interface quickly becomes boilerplate-heavy. A useful model should make such propagation explicit and compositional, without collapsing these capabilities into ambient authority.

Third, a capability system should support *simulation and testing* by treating capabilities as abstract interfaces rather than hidden ambient mechanisms. When capabilities remain explicit program values, the same program structure can be exercised against restricted, attenuated, or mocked capabilities. Existing capability systems do not always make that form of substitution natural at the level of the model.

1.2 Contextual Capabilities

This paper proposes a mathematical model of *contextual capabilities*. The basic idea is that capabilities are ordinary typed values that can be explicitly provided or rebound for a context. Code deep in a call chain may then use them without every intermediate function carrying them as ordinary parameters. Their use is tracked statically, but their availability is determined by surrounding program context rather than by ambient authority.

1.3 Contribution

We present λ^{CC} , a minimal calculus of contextual capabilities. Its contribution is to combine explicit capability values, static capability tracking, and a higher-order operational account in one verified formalism. The resulting model supports reasoning about restricted execution, capability-sensitive interfaces, authority attenuation, and the safe execution of untrusted code. It is intended as a design model for new secure programming languages, rather than as a complete implementation strategy by itself.

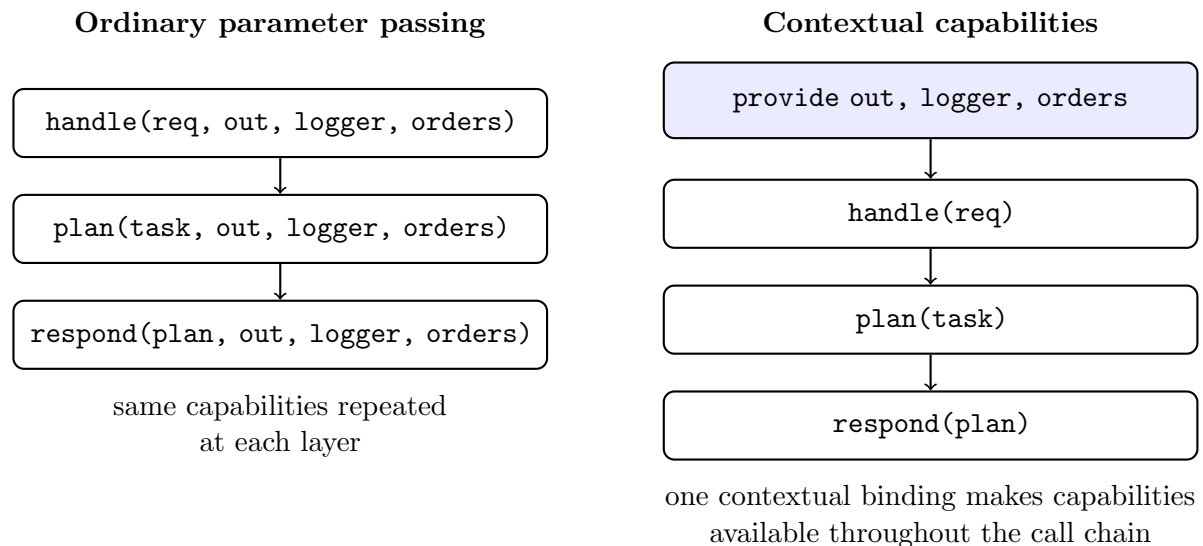


Figure 1: Ordinary parameter plumbing versus contextual capability propagation

2 Contextual Capabilities

The easiest way to understand contextual capabilities is to start from a mechanism every reader already knows: ordinary function parameters. If an operation needs an output channel, a logger, and a user-scoped orders service, one direct design is to pass all of them explicitly:

```
handle(req, out, logger, orders)
```

and then continue threading those same parameters through helpers that may not use them directly:

```
handle(req, out, logger, orders)
→ plan(task, out, logger, orders)
→ respond(plan, out, logger, orders)
```

This style is explicit and testable, but it is also noisy. Intermediate functions often carry capabilities only so later code can eventually use them.

Contextual capabilities keep the good part of parameter passing and remove that plumbing cost. A capability is still an ordinary typed value, but it may be *bound for a context* rather than passed through every signature. Code deeper in the call chain can then use that capability because the surrounding context makes it available. In that sense, capabilities behave like remotely bound parameters that propagate through the call chain automatically, while remaining statically tracked and explicitly controllable.

Figure 1 is the central intuition. Contextual capabilities do not make authority ambient. They make authority available through explicit program context. A caller may bind an output capability for a single execution. Trusted code may rebind a user-scoped orders capability for one region of computation. A closure may package a capability behind a narrower interface. In each case, the capability remains an ordinary value; what changes is only how it becomes available to the code that uses it.

The Challenge. The challenge is to give this convenience a sound static account. A capability should be usable only when some program context has explicitly provided it, even in the presence of deep and complex transitive call chains and first-class functions.

Lambdas make that harder. A closure may carry capability usage transitively: it may use a capability itself, or call another function that uses it. A formal model therefore has to track not

Names	$n \in \mathbb{N}$
Capability names	c, d
Capabilities	$\varphi ::= \emptyset \mid \{c\} \mid \varphi_1 \cup \varphi_2$
Types	$T ::= \text{Nat} \mid T_1 \xrightarrow{\varphi} T_2$
Terms	$t ::= n \mid x \mid \mathbf{c} \mid \lambda(x:T_1, \varphi).t \mid t_1 t_2$ $\mid \mathbf{with} \ c = e_1 \ \mathbf{in} \ e_2 \mid \mathbf{allow} \ \varphi \ \mathbf{in} \ e$
Values	$v ::= n \mid \langle \lambda x.t, \rho, \sigma \rangle$

Figure 2: Syntax of λ^{CC}

only direct capability access, but also capability use delayed through closure capture and later invocation.

The calculus λ^{CC} provides a formal foundation for statically checked contextual capabilities.

3 The λ^{CC} Calculus

λ^{CC} is a small extension of the simply-typed lambda calculus with three capability constructs: capability access $\mathbf{c}$, capability binding $\mathbf{with} \ c = e_1 \ \mathbf{in} \ e_2$, and capability restriction $\mathbf{allow} \ \varphi \ \mathbf{in} \ e$.

We assume the reader is already comfortable with standard formal-language notions such as syntax, typing judgments, and big-step evaluation for the simply-typed lambda calculus; Pierce’s *Types and Programming Languages* is a standard reference [14].

Figure 2 gives the reader-facing surface syntax. Function syntax uses brace notation, while function types place the designated capability set above the arrow: $\lambda(x:T_1, \varphi).t$ denotes a function that receives the call-site capabilities in φ , and $T_1 \xrightarrow{\varphi} T_2$ is the corresponding function type. The types of the capabilities named in φ are defined in the global capability declaration Π .

3.1 Semantics

Evaluation uses a value environment ρ for term variables and a capability environment σ for capabilities. The evaluation judgment

$$\langle\langle t, \rho, \sigma \rangle\rangle \Downarrow v$$

means that t evaluates to v under those environments.

The rules are shown in Figure 3. Capability lookup reads from σ . Capability binding evaluates a body under an updated capability environment. Restriction runs a term in a restricted capability environment. The main rule is closure creation: evaluating $\lambda(x:T, \varphi).t$ stores $\sigma \ominus \varphi$, not the full capability environment.

This distinction corresponds to two different use cases. Some capabilities are meant to come from the function’s creation context and therefore should be captured in the closure. Others are meant to come from the call site and therefore should be supplied by the caller at invocation time.

3.2 Type System and Capability Tracking

Figure 4 presents the full system. The type system assigns each term both a type and a capability set. The type describes the value produced; the capability set is a static upper bound on the

$$\begin{array}{c}
\frac{}{\langle\!\langle n, \rho, \sigma \rangle\!\rangle \Downarrow n} \text{E-CONST} \quad \frac{\rho(x) = v}{\langle\!\langle x, \rho, \sigma \rangle\!\rangle \Downarrow v} \text{E-VAR} \quad \frac{\sigma(c) = v}{\langle\!\langle \mathbf{c}, \rho, \sigma \rangle\!\rangle \Downarrow v} \text{E-CVAR} \\
\\
\frac{}{\langle\!\langle \lambda(x:T, \varphi). t, \rho, \sigma \rangle\!\rangle \Downarrow \langle \lambda(x:T, \varphi). t, \rho, \sigma \ominus \varphi \rangle} \text{E-ABS} \\
\\
\frac{\langle\!\langle t_1, \rho, \sigma \rangle\!\rangle \Downarrow \langle \lambda(x:T, \varphi). t, \rho_1, \sigma_1 \rangle \quad \langle\!\langle t_2, \rho, \sigma \rangle\!\rangle \Downarrow v_2 \quad \langle\!\langle t, (\rho_1, x \mapsto v_2), (\sigma \uplus \sigma_1) \rangle\!\rangle \Downarrow v_3}{\langle\!\langle t_1 t_2, \rho, \sigma \rangle\!\rangle \Downarrow v_3} \text{E-APP} \\
\\
\frac{\langle\!\langle e_1, \rho, \sigma \rangle\!\rangle \Downarrow v_1 \quad \langle\!\langle e_2, \rho, \sigma[c \mapsto v_1] \rangle\!\rangle \Downarrow v_2}{\langle\!\langle \mathbf{with } c = e_1 \text{ in } e_2, \rho, \sigma \rangle\!\rangle \Downarrow v_2} \text{E-WITH} \quad \frac{\langle\!\langle e, \rho, \sigma|_{\varphi} \rangle\!\rangle \Downarrow v}{\langle\!\langle \mathbf{allow } \varphi \text{ in } e, \rho, \sigma \rangle\!\rangle \Downarrow v} \text{E-ALLOW}
\end{array}$$

Figure 3: Evaluation rules for λ^{CC}

$$\begin{array}{c}
\frac{}{\Pi, \Gamma \vdash n : \text{Nat} ! \emptyset} \text{T-CONST} \quad \frac{\Gamma(x) = T}{\Pi, \Gamma \vdash x : T ! \emptyset} \text{T-VAR} \quad \frac{c \in \Pi \quad \Pi(c) = T}{\Pi, \Gamma \vdash \mathbf{c} : T ! \{\mathbf{c}\}} \text{T-CVAR} \\
\\
\frac{\Pi, \Gamma, x:T_1 \vdash t : T_2 ! \varphi'}{\Pi, \Gamma \vdash \lambda(x:T_1, \varphi). t : T_1 \xrightarrow{\varphi} T_2 ! \varphi' \setminus \varphi} \text{T-ABS} \quad \frac{\Pi, \Gamma \vdash t_1 : T_1 \xrightarrow{\varphi} T_2 ! \varphi_1 \quad \Pi, \Gamma \vdash t_2 : T_1 ! \varphi_2}{\Pi, \Gamma \vdash t_1 t_2 : T_2 ! \varphi \cup \varphi_1 \cup \varphi_2} \text{T-APP} \\
\\
\frac{\Pi(c) = T_1 \quad \Pi, \Gamma \vdash e_1 : T_1 ! \varphi_1 \quad \Pi, \Gamma \vdash e_2 : T_2 ! \varphi_2}{\Pi, \Gamma \vdash \mathbf{with } c = e_1 \text{ in } e_2 : T_2 ! \varphi_1 \cup (\varphi_2 \setminus \{\mathbf{c}\})} \text{T-WITH} \\
\\
\frac{\Pi, \Gamma \vdash e : T ! \varphi' \quad \varphi' \subseteq \varphi}{\Pi, \Gamma \vdash \mathbf{allow } \varphi \text{ in } e : T ! \varphi'} \text{T-ALLOW} \quad \frac{\Pi, \Gamma \vdash t : T ! \varphi \quad c \in \Pi}{\Pi, \Gamma \vdash t : T ! \varphi \cup \{\mathbf{c}\}} \text{T-SUB}
\end{array}$$

Figure 4: Typing rules for λ^{CC}

authority the term may use. The judgment

$$\Pi, \Gamma \vdash t : T ! \varphi$$

means that term t has type T and may access capabilities in φ .

T-CVAR is the rule that actually uses a capability: accessing $\mathbf{c}$ adds exactly c to the term's capabilities. T-ABS is the central rule. If the function body may use capabilities φ' , and the lambda marks φ as caller-provided, then the lambda expression itself requires only $\varphi' \setminus \varphi$ at its definition site. In other words, capabilities listed in φ are expected from the call site rather than from closure creation. The function type records the caller-provided capability set φ above the arrow.

T-APP restores caller responsibility: applying a function of type $T_1 \xrightarrow{\varphi} T_2$ contributes φ to the application's capabilities. This is the static counterpart of the runtime semantics, where the closure does not capture the capabilities in φ and the caller must supply them at application time.

T-WITH binds a value to capability c and masks that requirement in the body, so uses of c inside e_2 do not remain obligations of the outer context. T-ALLOW turns dynamic restriction into a static contract: if e checks with capability set φ' and $\varphi' \subseteq \varphi$, then restricting execution to φ cannot remove a capability that the term needs. T-SUB is standard weakening.

3.3 Security Properties

At a high level, λ^{CC} guarantees that

- capability annotations are trustworthy upper bounds on capability use, and
- restricted computations cannot rely on capabilities outside the declared boundary.

If a term is typed with capabilities φ , then evaluation may access only capabilities in φ from the enclosing environment.

Confinement. Because capability provisioning and access is explicit and checked, well-typed code is confined to the provided capabilities.

4 Soundness Theorem

This section explains how the soundness of the capability model is established.

4.1 Main Theorem

Theorem 4.1 (Soundness). *For any capability declaration Π , term t , and type T :*

$$\Pi, \cdot \vdash t : T ! \emptyset \implies \exists v. \langle\langle t, \cdot, \cdot \rangle\rangle \Downarrow v$$

In words: a closed term typed with empty capabilities evaluates successfully with no capabilities provided.

4.2 Why This Is Enough

This theorem establishes the central static guarantee of the calculus: the capability annotation in the type system is sound. If the type system says a term needs no capabilities, then evaluation agrees.

The more general statement, used in the proof, is that a term typed with capability set φ evaluates successfully under any capability environment that supplies φ .

4.3 Proof Strategy

The proof uses logical relations, a standard technique for semantic type soundness arguments [15]. Rather than reasoning directly about all evaluations, we define semantic predicates describing when:

- a value satisfies a type,
- a value environment satisfies a typing environment, and
- a capability environment satisfies a capability set.

The central proof obligation is then the familiar one: well-typed terms in satisfying environments evaluate to satisfying values.

4.4 The Fundamental Lemma

Definition 4.2 (Logical Relations). *For a capability declaration Π , the proof defines three mutually supporting semantic predicates.*

$$\begin{aligned} \mathcal{V}[[T]]_{\Pi}(v) & \text{ means } v \text{ is a well-behaved semantic value of type } T, \\ \mathcal{E}[[\Gamma]]_{\Pi}(\rho) & \text{ means } \rho \text{ maps every } x:T \in \Gamma \text{ to some } v \text{ with } \mathcal{V}[[T]]_{\Pi}(v), \\ \mathcal{C}[[\varphi]]_{\Pi}(\sigma) & \text{ means } \sigma \text{ supplies every } c \in \varphi \text{ with a value satisfying } \Pi(c). \end{aligned}$$

*The value relation is defined by recursion on types. Natural-number values satisfy **Nat**. A closure satisfies $T_1 \xrightarrow{\mathcal{L}} T_2$ when, for every argument satisfying T_1 and every caller capability environment satisfying φ , evaluating the closure body under the extended value environment and merged capability environment produces a result satisfying T_2 .*

Lemma 4.3 (Fundamental Lemma). *For any $\Pi, \Gamma, \rho, \sigma, t, T$, and φ :*

$$\begin{array}{l} \mathcal{E}[\Gamma]_{\Pi}(\rho) \wedge \\ \mathcal{C}[\varphi]_{\Pi}(\sigma) \wedge \\ \Pi, \Gamma \vdash t : T ! \varphi \end{array} \implies \exists v. \langle t, \rho, \sigma \rangle \Downarrow v \wedge \mathcal{V}[T]_{\Pi}(v)$$

This lemma is the real engine of the proof. The theorem follows immediately by instantiating it with empty environments and empty capabilities.

4.5 Mechanization

The development is mechanized in Coq. The mechanization covers the definitions, auxiliary lemmas on capability environments, the fundamental lemma, and the final soundness theorem.

For this paper, the important point is not merely that the theorem has been checked by a proof assistant. The machine-checked proof confirms the mathematical soundness of the capability model.

5 Case Study: Confining LLM-Generated Code

The model is implemented in Jo, a new secure programming language (<https://jo-lang.org>). This section uses Jo’s surface syntax to show how the model applies to one representative problem: safely executing code generated by a large language model.

Jo follows the object-capability discipline: dangerous operations are available only through explicit capability values. The capability system adds static checking to that discipline. In Jo, `with` provides a capability value to a nested scope, `allow` restricts which capabilities may be used in that scope, and `receives` records an explicit capability requirement in a function signature.

5.1 The Problem

A platform may want generated code to analyze a user’s data and emit a result, while preventing it from reading arbitrary files, making network requests, calling unrestricted tools, or accessing other users’ data. The generated code is useful only if it can perform some actions, but those actions must be narrow and reviewable.

The naive design is to give generated code a broad runtime object, database handle, or tool interface and rely on conventions inside the generated code. That is not confinement; it is trust. Even if the generated code is asked to use only one user’s orders, a broad database capability gives it more authority than the policy intends.

5.2 The Jo Design

The framework is split into two libraries. The *interface library* is the library against which generated code is checked. It contains data types, capability declarations, and the expected entry-point signature, but no FFI or other ambient authority. The *runtime library* is trusted code. It may use FFI, databases, filesystem access, or other real services, but it uses that authority only to construct the capability values supplied to the generated code.

Figure 5 is the operational boundary. The interface library is the contract between the two sides. Generated code is not compiled against the runtime library, so it cannot name its FFI-backed operations. Instead, generated code is compiled against the interface library, while the trusted runtime library implements that interface and supplies concrete capability values after generated code has been checked.

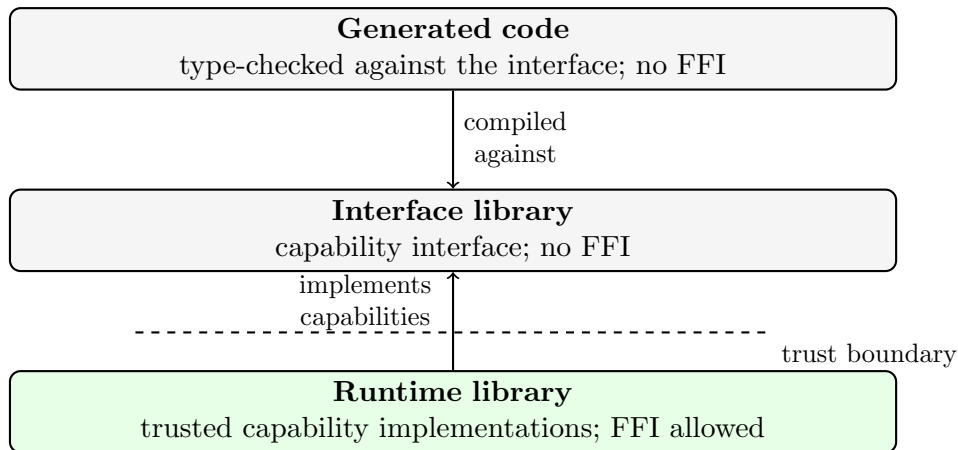


Figure 5: Jo framework split for confining generated code. Only the runtime library below the trust boundary needs to be trusted.

In this example, the only input capability is `GetOrders`, a function that returns orders for the current user, and the only output capability is `IO.stdout`. The generated entry point must declare that it receives exactly those capabilities.

```

1 param GetOrders: Int => List[Order] // lastDays -> orders
2
3 defer def aiMain(): Unit receives GetOrders, IO.stdout

```

The framework constructs the actual capability values. It may internally use broad authority such as a database connection, the current user identity, and a mutable output buffer. Those values remain in trusted code. The value given to generated code is an attenuated function that already incorporates the user restriction.

```

1 def frameworkMain() =
2   val db = connect("orders.db")
3   val userId = currentUser()
4   val restricted = (days: Int) => db.ordersFor(userId, days)
5
6   val output: mutable.List[String] = []
7   val buffer = (s: String) => output += s
8
9   allow none in
10    with GetOrders = restricted, IO.stdout = buffer in aiMain()

```

The generated program is checked against the interface it is allowed to use. It can read recent orders and write a summary, but it has no name for the database, filesystem, network, or broader tool environment.

```

1 def aiMain(): Unit receives GetOrders, IO.stdout =
2   val orders = GetOrders(30)
3   summarize(orders)

```

5.3 Why This Confines Authority

The important point is where authority lives. The framework may possess broad authority, but it does not pass that authority to generated code. Instead, it derives a narrower capability,

`restricted`, and binds that value as `GetOrders`. This is authority attenuation: the generated code can ask for orders, but only through a function that has already fixed the user scope.

`allow none` is also significant. It says that the body being run may not rely on ambient or already-in-scope capabilities beyond those explicitly provided by the nested `with`. If generated code, or code reachable from it, requires some additional capability, the compiler rejects the program. This gives the framework a static check that all direct and transitive capability requirements have been mediated.

This is the practical role of the captured-versus-caller-provided distinction in the calculus. Trusted closures may capture broad internal state in order to build narrow interfaces, but generated code receives only the caller-provided capabilities listed in its signature and supplied at the call site. Captured authority is used to implement attenuation; caller-provided authority is used to preserve control over invocation.

The result is a design in which generated code can be useful without being trusted with the whole runtime. It receives exactly the capabilities named by its interface, and the type system checks that no additional capability is used.

6 Related Work

The most relevant prior work falls into three groups.

6.1 Capability Disciplines Without Static Tracking

Capability systems from Dennis and Van Horn onward [1] established the core authority model: capabilities are explicit, unforgeable references to authority. Miller’s object-capability account [2, 3] and languages such as E [4], Joe-E [5], and Newspeak [6] demonstrate that this discipline scales to realistic programming languages.

What these systems give is a strong semantic and engineering story about authority flow. What they generally do not give is a small static account of which capabilities a term may use, nor a compact proof-oriented model of fine-grained call-site capability provision.

6.2 Static Requirement Tracking

Lucassen and Gifford [11] and Talpin and Jouvelot [12] introduced influential frameworks for tracking program requirements in types. λ^{CC} follows that general direction, but applies it specifically to capability access.

The key difference is not merely that λ^{CC} tracks capabilities. It is that it tracks them in a setting where some capabilities remain under call-site control rather than being uniformly captured or manually threaded. In that sense, the model is also close in spirit to coeffects: it describes what a computation requires from its context [13].

6.3 Verified Capability Reasoning

Swasey *et al.* [10] verify object-capability patterns in a richer language using logical relations. Their work is closer in proof style than in modeling goal: they verify specific patterns in a realistic setting, whereas λ^{CC} isolates a minimal core calculus with global capability soundness.

Devriese and Piessens [9] show, in a different security setting, that static reasoning can establish strong semantic guarantees. That result is not about capabilities, but it supports the general thesis that small static models can carry real security meaning.

7 Conclusion

λ^{CC} is a small calculus aimed at three gaps in prior capability work: the lack of a compact static account of capability use, the lack of a clean formal model for fine-grained capability propagation through higher-order code, and the lack of a corresponding foundation for simulation and testing with explicit capability values.

The model addresses these gaps with contextual capabilities: capabilities that are tracked statically, yet may still be supplied by the calling context rather than captured unconditionally. This makes it possible to reason about two important patterns at once:

- packaging attenuated authority into narrow interfaces, and
- preserving caller control over invocation-time capabilities such as output channels or tool bindings.

The soundness theorem shows that capability annotations are sound with respect to evaluation. If the type system says a term uses no capabilities, then evaluation succeeds with no capabilities provided. More generally, the model supports rigorous reasoning about authority restriction, capability containment, and higher-order confinement.

The result is a verified semantic core for statically checked contextual capabilities, especially relevant when executing untrusted or LLM-generated code. Its main purpose is to guide the design of new secure programming languages that make capability use explicit, statically checkable, and controlled by program context.

Resource bounds and covert channels are not addressed by this core calculus; they must be handled by other means, such as runtime resource enforcement and side-channel controls.

References

- [1] J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [2] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, 2006.
- [3] M. S. Miller, K.-P. Yee, and J. Shapiro. Capability myths demolished. Technical Report SRL2003-02, Johns Hopkins University, 2003.
- [4] M. S. Miller, E. D. Tribble, and J. Shapiro. Concurrency among strangers: Programming in E as plan coordination. In *Symposium on Trustworthy Global Computing*, pages 195–229. Springer, 2005.
- [5] A. Mettler, D. Wagner, and T. Close. Joe-E: A security-oriented subset of Java. In *Network and Distributed System Security Symposium (NDSS)*, 2010.
- [6] G. Bracha and D. Ungar. Mirrors: Design principles for meta-level facilities of object-oriented programming languages. In *OOPSLA*, pages 331–344, 2004.
- [7] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [8] N. Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- [9] D. Devriese and F. Piessens. Noninterference through secure multi-execution. In *IEEE Symposium on Security and Privacy*, pages 109–124, 2010.

- [10] D. Swasey, D. Garg, and D. Dreyer. Robust and compositional verification of object capability patterns. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA):1–26, 2017.
- [11] J. M. Lucassen and D. K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 47–57, 1988.
- [12] J.-P. Talpin and P. Jouvelot. Polymorphic type, region and effect inference. *Journal of Functional Programming*, 2(3):245–271, 1992.
- [13] T. Petricek, D. Orchard, and A. Mycroft. Coeffects: A calculus of context-dependent computation. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, pages 123–135, 2014.
- [14] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [15] B. C. Pierce, editor. *Advanced Topics in Types and Programming Languages*. MIT Press, 2005.